

FWT

状压 dp 教程

目录 I

1 集合幂级数

2 卷积算法

3 例题

目录

1 集合幂级数

2 卷积算法

3 例题

集合幂级数

我们使用状压 dp 的时候，数组下标一般都是一个集合。对于这种下标为集合的序列 f_S ，我们定义它的生成函数，也就是集合幂级数为：

$$F(x) = \sum_{S \subseteq U} x^S f_S$$

其中 U 是全集，即 $U = \{1, 2, 3, \dots, n\}$ 。

但是不同于生成函数的是，这个东西本身的确没有什么代数含义， x^S 仅是占位符号，只不过是单纯的用更简单的方式描述一个序列或者 dp 数组罢了。

集合幂级数的卷积

dp 时对数组做的一些操作往往可以刻画为一些函数的变化操作，其中比较重要而不平凡的操作是卷积。

定义集合幂级数 f_S, g_S 的卷积为：
$$h_S = \sum_{L, R \subseteq U} f_L g_R [c(L, R) = U].$$

其中 c 函数一般是一种位运算操作。

或者说也可以这么考虑卷积， f_L, g_R 的乘积贡献到 $[x^{c(L, R)}]H(x)$ 上。就和序列卷积 f_i, g_j 的乘积贡献到 $[x^{i+j}]H(x)$ 上类似。

使用 FWT 可以将一般的卷积做到更优秀的复杂度。

接下来只讨论或，异或操作。

目录

1 集合幂级数

2 卷积算法

3 例题

并卷积

考虑集合并卷积。

根据**位运算在每一位的独立性**，我们考虑分治求解卷积。

设 $f = f^- + x^{\{n\}} \cdot f^+$, $g = g^- + x^{\{n\}} \cdot g^+$ (注意 $\cdot$ 表示集合并卷积，即两个集合贡献到它们的并)。即将第 n 位单独提出来， f^- 表示 f 不包含元素 n 的所有集合对应的集合幂级数，或者说 $f_0 x^0 \sim f_{2^{n-1}-1} x^{2^{n-1}-1}$ 单独提取出来，即

$$f^- = \sum_{n \notin S} f_S x^S = \sum_{0 \leq i < 2^{n-1}} f_i x^i.$$

而 f^+ 表示 f 包含元素 n 的所有集合对应的集合幂级数，同时将这些集合中的元素 n 去掉，但系数保留，即

$$f^+ = \sum_{n \in S} f_S x^{S \setminus \{n\}} = \sum_{2^{n-1} \leq i < 2^n} f_i x^{i-2^{n-1}}.$$

显然， $f \cdot g = (f^- + x^{\{n\}} \cdot f^+) \cdot (g^- + x^{\{n\}} \cdot g^+)$ 。由于 $x^{\{n\}} \cdot x^{\{n\}} = x^{\{n\}}$ ，因此

$$f \cdot g = f^- \cdot g^- + x^{\{n\}} \cdot ((f^- + f^+) \cdot (g^- + g^+) - f^- \cdot g^-).$$

因此，上述分治做法的时间复杂度为 $\mathcal{O}(n2^n)$ 。

并卷积

注意到我们将问题分成了两次不包含 n 的两个集合幂级数的集合并卷积。求解 $T(n) = 2T(\frac{n}{2}) + \mathcal{O}(n)$, 得到 $T(n) = \mathcal{O}(n \log n)$ 。

FWT

观察分治做法，可以发现在后半部分，我们相当于先将所有 f_i ($2^{n-1} \leq i < 2^n$) 加上 $f_{i-2^{n-1}}$ ，相乘后再减去 $f_{i-2^{n-1}}$ 相乘之后的结果。前面一部分给我们的感觉是**将所有第 n 位为 0 的项前的系数加到其对应的第 n 位为 1 的项前的系数上**，后面一部分则是**进行了这种变换的两个集合幂级数的对应系数相乘**（因为递归到最底层是对应系数相乘）后，将所有第 n 位为 1 的项前的系数减去其对应的第 n 位为 0 的项前的系数。于是不妨先做完这个高位前缀和，按位相乘之后，再减回去。

于是设序列 f 的莫比乌斯变换 $\text{FWT}(f) = \hat{f} = \sum_{S \subseteq U} x^S \sum_{T \subseteq S} f_T$ 。

以及逆变换 $\text{IFWT}(f) = \hat{f} = \sum_{S \subseteq U} x^S \sum_{T \subseteq S} f_T (-1)^{|S|-|T|}$ 。

那么 $h = \text{IFWT}(\text{FWT}(f) \times \text{FWT}(g))$ ，其中乘法是按位乘，就和 FFT/NTT 做的事情一模一样。

FWT

下面严谨证明一下，我们需要证明两点： $\text{IFWT}(\text{FWT}(f)) = f$ ，以及 $\text{FWT}(h) = \text{FWT}(f) \times \text{FWT}(g)$ 。

$$\sum_{T \subseteq S} (-1)^{|S \setminus T|} \left(\sum_{U \subseteq T} f_U \right) = \sum_{U \subseteq S} f_U \sum_{U \subseteq T \subseteq S} (-1)^{|S \setminus T|} = \sum_{U \subseteq S} f_U \sum_{V \subseteq S \setminus U} (-1)^{|V|} = f_S$$

据定义， $h_S = \sum_L \sum_R [L \cup R = S] f_L g_R$ ，因此 $\hat{h}_S = \sum_L \sum_R [L \cup R \subseteq S] f_L g_R$ 。由于 $[L \subseteq S][R \subseteq S]$ 等价于 $[L \cup R \subseteq S]$ ，故 $\hat{h}_S = \sum_{L \subseteq S} f_L \sum_{R \subseteq S} g_R = \hat{f}_S \hat{g}_S$ ，证毕。

代码实现也相当简单，直接做高维前缀和，以及高维差分即可。

FWT

$\text{IFWT}(\text{FWT}(f)) = f$ 的另一种理解：考虑到高维前缀和在每一个 bit 上做的事情可以看作是矩阵乘法：

$[f_0, f_1] \times \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} = [f'_0, f'_1]$ ，差分同理，而前缀和的矩阵和差分矩阵互为逆矩阵，所以自然成立。

而巧合的一点是，这个矩阵变换之后的 FWT 数组按位乘恰好就是或卷积所需要的。
我们可以把 FWT 变换后的数组视作为集合幂级数的点值序列。

例题

给你一个序列，值域在 $[0, 2^n)$
对于每一个 $S \in [0, 2^n)$ 以及每一个 $i \in [0, n]$ ，计算 $|\text{Sand}_{a_j}| = i$ 的数量。
 $1 \leq n \leq 20$ 。

我前天的 idea

给出一个二分图，左部右部都有 2^n 个点，左部点中只有前 n 个点的度数可以不大于 2^n ，给出这 n 个点的连边情况。

分别求这张图的二分图的 $2^n, 2^n - 1, \dots, 2^n - n + 1$ 匹配数对 p 取模的答案。

$1 \leq n \leq 20, 10^9 \leq p \leq 1.1 \times 10^9$ 。

异或卷积

考虑集合异或卷积。

类似地，利用位运算在每一维独立的性质，考虑分治。

设 $f = f^- + x^{\{n\}} \cdot f^+$ ， $g = g^- + x^{\{n\}} \cdot g^+$ ，类似集合并卷积的推导，我们有

$$\begin{aligned} fg &= (f^- + x^{\{n\}} f^+)(g^- + x^{\{n\}} g^+) \\ &= (f^- g^- + f^+ g^+) + x^{\{n\}} (f^- g^+ + f^+ g^-) \end{aligned}$$

如果直接拆成四个子卷积，时间复杂度仍然是 $\mathcal{O}(4^n)$ 。考虑计算 $p = (f^- + f^+)(g^- + g^+)$ 以及 $q = (f^- - f^+)(g^- - g^+)$ （当然， $(f^+ - f^-)(g^+ - g^-)$ 也可以，可以说明两者是等价的）。那么

$$fg = \frac{p+q}{2} + x^{\{n\}} \frac{p-q}{2}。时间复杂度 \mathcal{O}(n2^n)。$$

FWT

把这个东西换成上文中的非递归就相当困难了，但思路也是一样的，依旧考虑求出一个 FWT 变换的方法，使得按位乘后的序列就是目标序列的 FWT 序列，再逆变换回去。

我们进行一步转化： $[S = \emptyset] = \frac{1}{2^n} \sum_{T \subseteq 2^U} (-1)^{|S \cap T|}$ 。若 $S \neq \emptyset$ ，则选出 S 中任意一个元素 x ，包含 x

的 T 和不包含 x 的 T 一一对应且贡献相反。若 $S = \emptyset$ 显然成立。

注意到 $L \oplus R = S$ 等价于 $L \oplus R \oplus S = \emptyset$ ，且 $(L \oplus R) \cap S = (L \cap S) \oplus (R \cap S)$ ，所以

$$\begin{aligned} h_S &= \sum_{L \in 2^U} \sum_{R \in 2^U} [L \oplus R = S] f_L g_R \\ &= \frac{1}{2^n} \sum_{L \in 2^U} \sum_{R \in 2^U} \sum_{T \subseteq 2^U} (-1)^{|(S \oplus L \oplus R) \cap T|} f_L g_R \\ &= \frac{1}{2^n} \sum_{T \subseteq 2^U} (-1)^{|S \cap T|} \left(\sum_{L \in 2^U} (-1)^{|L \cap T|} f_L \right) \left(\sum_{R \in 2^U} (-1)^{|R \cap T|} g_R \right) \end{aligned}$$

FWT

因此, 考虑令 $\hat{f}_S = \sum_{T \subseteq 2^U} f_T (-1)^{|S \cap T|}$, 它被称为 f 的**沃尔什变换**, $\hat{f}$ 也记作 $\text{FWT}(f)$, 因为我们用快速沃尔什变换 (Fast Walsh-Hadamard Transform) 求解沃尔什变换。

同理, 根据 $[S = \emptyset] = \frac{1}{2^n} \sum_{T \subseteq 2^U} (-1)^{|S \cap T|}$, 我们定义 $\hat{f}$ 的**沃尔什逆变换**为 $f_S = \frac{1}{2^n} \sum_{T \subseteq 2^U} \hat{f}_T (-1)^{|S \cap T|}$ 。

f 也记作 $\text{IFWT}(\hat{f})$, 因为我们用快速沃尔什逆变换 (Inverse Fast Walsh-Hadamard Transform) 求解沃尔什逆变换。

$$\begin{aligned}
 f_S &= \frac{1}{2^n} \sum_{T \subseteq 2^U} \sum_{X \subseteq 2^U} (-1)^{|S \cap T|} f_X (-1)^{|T \cap X|} \\
 &= \sum_{X \subseteq 2^U} f_X \frac{1}{2^n} \sum_{T \subseteq 2^U} (-1)^{|(S \oplus X) \cap T|} \\
 &= \sum_{X \subseteq 2^U} f_X [S \oplus X = \emptyset] \\
 &= f_S
 \end{aligned}$$

FWT

设当前考虑第 j 位，则对于两个除了第 j 位以外均相同的位置 p, q ($p + 2^{j-1} = q$)，由于 p 这一位是 0，由于 0 和 0/1 的与均为 0，因此贡献了 0 个 -1 ，故 $f_p \leftarrow f_p + f_q$ 。同理，因为 $1 \wedge 0 = 0$ ， $1 \wedge 1 = 1$ ，可以推出 $f_q \leftarrow f_p - f_q$ 。因此对每一位的每两对这样的 p, q 做如上变换，即可得到 f 的沃尔什变换。这相当于依次把每一位 j （每一个元素）放到

$$f_i = \sum_{0 \leq k < n} (-1)^{\text{popcount}(i \text{ and } k)} f_k$$

的考虑范围内。

子集卷积

将集合之间的二元运算定义为**不相交集合并**，即

$$x^L \cdot x^R = \begin{cases} 0 & (L \cap R \neq \emptyset) \\ x^{L \cup R} & (L \cap R = \emptyset) \end{cases},$$

我们得到了**子集卷积**的形式

$$h_S = \sum_{L \in 2^U} \sum_{R \in 2^U} [L \cup R = S][L \cap R = \emptyset] f_L g_R$$

它和集合并卷积的唯一区别在于两个集合不能有交。暴力枚举子集，时间复杂度 $\mathcal{O}(3^n)$ 。

注意到当 $L \cup R = S$ 时， $L \cap R = \emptyset$ 的**充要条件**是 $|L| + |R| = |S|$ 。因此，我们设 $f_{i,S} = [|S| = i]f_S$ ， g 同理，令 $H_i = \sum_{j+k=i} f_j \cdot g_k$ 。这里 H_i 表示一个集合幂级数而非集合幂级数 H 的第 i 项， $\cdot$ 表示集合并卷积。

子集卷积

根据性质我们有 $h_S = H_{|S|, S}$, 因为只有所有大小之和为 $|S|$ 的 L, R 才会贡献到 $H_{|S|}$ 上, 而取 $h_{|S|}$ 位置 S 上的值保证了贡献到该位置上的集合 L, R 的并为 S , 因此符合限制。

将等式两边同时取 FMT, 得到 $\hat{H}_i = \sum_{j+k=i} \hat{f}_j \cdot \hat{g}_k$, 这里 $\cdot$ 表示对应位置相乘。因此, 我们首先

$\mathcal{O}(n^2 2^n)$ 计算出每个 f_j 和 g_k 的莫比乌斯变换 (共有 n 个集合幂级数, 每做一次 FMT 需要 $\mathcal{O}(n 2^n)$ 的时间), 然后 $\mathcal{O}(n^2 2^n)$ 按式子计算出 $\hat{H}_i$ (共要做 n^2 次对应位置相乘, 每次需要 $\mathcal{O}(2^n)$ 的时间), 最后再将每个 $\hat{H}_i$ 逆变换回来即可。三个部分时间复杂度均为 $\mathcal{O}(n^2 2^n)$, 因此这也是总时间复杂度。

目录

1 集合幂级数

2 卷积算法

3 例题

例题

AT_abc212_h, AT_arc100_c, P4221, CF914G, P5387, AT_abc288_g